

PROJECT 1 — MARKET DATA LAKE

Value Investing Challenge • RA1

EVERY PREDICTION MUST BE RECORDED BEFORE WE KNOW THE OUTCOME.

Una predicción no es "esta acción va a subir". Una predicción es: *"dada la información disponible hoy, mi sistema estima un 67% de probabilidad de que esta acción alcance al menos +10% en los próximos 3 meses"*. Esa diferencia — probabilidad explícita, horizonte explícito, fecha de emisión explícita — es lo que permite evaluar el sistema en lugar de discutir opiniones.

1. Introducción del proyecto

Trabajáis como Data Engineers construyendo la capa de datos de un sistema de inteligencia de inversión. No construís la web: la plataforma **Value Investing Challenge** ya existe y la mantiene el profesor. Vuestro sistema expone un endpoint HTTP; la plataforma lo consulta cada ciclo semanal, congela la respuesta de forma inmutable, espera a que la realidad ocurra y publica un leaderboard con todos los alumnos.

El proyecto es continuo a lo largo de los tres RA. Lo que construyáis aquí es el sustrato de todo lo demás: en RA2 el Data Lake alimenta un Data Warehouse y modelos de Machine Learning, en RA3 los agentes de IA consultan ese warehouse. Si el lake es pobre, el ML de RA2 no tiene nada que aprender y los agentes de RA3 alucinan números.

La tarea de predicción es idéntica en los tres RA. Cada ciclo (una semana) entregáis **Top 3 tickers por horizonte** para `1W`, `1M`, `3M` y `6M`, es decir **máximo 12 predicciones por ciclo**. Cada predicción responde a una única pregunta: dada solo la información disponible hoy, ¿qué probabilidad hay de que esta acción alcance al menos **+10%** de retorno dentro del horizonte? Una predicción es **hit** cuando `realized_return >= 0.10`.

En RA1 vuestro modelo se registra como `v1`: un scoring manual con pesos elegidos por vosotros. Los resultados de `v1` no se borran nunca. En RA2 subiréis `v2` (ML) y en RA3 `v3` (agentes), y la plataforma comparará las tres versiones para responder a las dos preguntas que estructuran el curso: ¿el ML batió al scoring manual? ¿los agentes batieron al ML?

2. Objetivos de aprendizaje

- Diseñar e implementar un pipeline de ingesta desde fuentes heterogéneas (APIs REST, CSV, JSON, texto) hacia almacenamiento columnar.

- Procesar datos financieros con Spark/PySpark y materializarlos en Parquet o Delta Lake.
- Definir una estrategia de particionado coherente con el patrón de lectura y justificarla con números.
- Implementar controles de calidad de datos: nulos, duplicados, rangos, zonas horarias, festivos de mercado, splits y dividendos.
- Construir un **Baseline Investment Score** como combinación lineal ponderada de features normalizadas, y justificar cada peso.
- Transformar un score de ranking en una probabilidad calibrada, entendiendo por qué no son lo mismo.
- Exponer el resultado como JSON válido conforme al contrato del Value Investing Challenge.

3. Arquitectura

```

Financial APIs / CSV / JSON / Text
      ↓
Python ingestion
      ↓
Spark
      ↓
Delta / Parquet
      ↓
DATA LAKE
  
```

Tres zonas lógicas dentro del lake, con nombres a vuestra elección (bronze/silver/gold, raw/clean/curated):

ZONA	CONTENIDO	FORMATO	MUTABILIDAD
Raw	Respuesta literal de la API, sin tocar	JSON/CSV comprimido	Append-only, nunca se sobrescribe
Clean	Tipado, deduplicado, timezone normalizada, splits ajustados	Parquet/Delta	Reescribible por partición
Curated	Features y <code>baseline_score</code> por ticker y fecha	Parquet/Delta	Reescribible por partición

Guardar la zona Raw literal no es burocracia: cuando en RA2 detectéis que un campo estaba mal interpretado, poder reprocesar sin volver a llamar a una API con límite de 25 requests/día es la diferencia entre arreglarlo en una hora y perder un ciclo entero.

4. Requisitos

Vosotros decidís proveedores de datos, qué campos ingerir, la estrategia de calidad, el particionado y qué señales construir. No se impone una API. Lo que sí es obligatorio:

1. **Ingesta reproducible**: un script o notebook que, ejecutado de nuevo, reconstruye el lake desde las fuentes o desde la zona Raw.
2. **Procesamiento con Spark/PySpark**, no solo pandas. Con 10 tickers pandas sobra; el objetivo es que demostréis el modelo de ejecución distribuida y el particionado.
3. **Almacenamiento columnar particionado** en Parquet o Delta.
4. **Controles de calidad ejecutables** que emitan un informe (filas leídas, filas rechazadas, nulos por columna, duplicados detectados).
5. **Baseline Investment Score** con pesos documentados y justificados.
6. **Endpoint GET /api/predictions** que devuelva JSON válido según el contrato, con `model_version: "v1"`.

Datos a los que aspirar

No todos son obligatorios. Precios de mercado, fundamentales, valoración, crecimiento, rentabilidad, deuda, flujo de caja, earnings, competidores, noticias, sentimiento y macro. Empezad por precios y fundamentales: sin esos dos no hay score.

Fuentes gratuitas reales y sus límites

FUENTE	QUÉ DA	LÍMITE REAL
yfinance	Precios, fundamentales básicos	No oficial, hace scraping, se rompe sin aviso cuando Yahoo cambia el HTML
Alpha Vantage	Precios, fundamentales, earnings	Free tier ~25 requests/día. Con 10 tickers agotáis el día en dos vueltas
Finnhub	Precios, earnings, news, sentiment	Free tier con rate limit por minuto y endpoints premium bloqueados
Financial Modeling Prep	Fundamentales, ratios	Free tier limitado y con histórico recortado
SEC EDGAR (companyfacts , full-text search)	Fundamentales oficiales US GAAP con fecha de publicación	Gratis y oficial. Requiere User-Agent identificable y respetar el rate limit
Nasdaq Data Link	Series macro y datasets varios	Muchos datasets migrados a pago

Los free tiers estrangulan. Eso os fuerza a **cachear en el lake en lugar de llamar a la API repetidamente**, que es exactamente el motivo por el que existe un data lake. Si vuestro notebook llama a Alpha Vantage cada vez que se ejecuta, no habéis construido un lake: habéis construido un cliente HTTP.

SEC EDGAR merece atención especial porque expone la **fecha de publicación (filed)** además del cierre del periodo fiscal. Eso os salvará en RA2, donde el look-ahead bias es criterio de evaluación.

Calidad de datos: los problemas que os van a pasar

- **Missing data:** un fundamental ausente no es cero. Decidid política explícita (excluir el ticker del ranking, imputar por mediana sectorial, propagar el último valor conocido) y documentadla.
- **Duplicate records:** la misma fecha ingerida dos veces por reintentos. Definid clave natural (`ticker` , `date`) y deduplicad con criterio explícito de qué versión gana.
- **Timezones:** mezclar `America/New_York` con UTC desplaza barras un día. Normalizad todo a una zona y guardad la decisión. Los `generated_at` del contrato exigen ISO-8601 **con offset**.
- **Market holidays:** NYSE no abre todos los días laborables. Un momentum de 5 sesiones no son 5 días de calendario. Usad calendario de mercado, no `date_range` de pandas.
- **Stock splits:** un split 10:1 sin ajustar produce un -90% de retorno falso. Usad series ajustadas o aplicad el factor vosotros.
- **Dividends:** el precio cae el día ex-dividendo. Para un umbral de +10% en 6M esto importa. Decidid si trabajáis con retorno de precio o retorno total y sed coherentes.
- **API quality:** la misma métrica difiere entre proveedores. Si mezcláis fuentes, fijad una como autoritativa por campo.

5. Tecnologías recomendadas

Python, Spark, PySpark, Parquet, Delta Lake. El almacenamiento debe poder ejecutarse **gratis**. **AWS no es obligatorio**.

OPCIÓN	COSTE	SETUP	SOBREVIVE A REINICIO DE SESIÓN	PARTICIONADO OBSERVABLE
Filesystem local	0	Minutos	Sí	Sí, <code>ls</code> muestra el árbol
Google Colab (disco efímero)	0	Inmediato	No, se pierde todo	Sí mientras dure la sesión
Colab + Google Drive montado	0	Minutos	Sí	Sí, pero I/O lento con muchos ficheros pequeños
MinIO en Docker local	0	~30 min	Sí	Sí, y además practicáis API S3 real
S3 / R2 / cualquier S3-compatible	Bajo, puede exceder free tier	~1 h con IAM	Sí	Sí, vía consola o CLI

Recomendación pragmática: desarrollad en local o MinIO, y si usáis Colab montad Drive desde el primer minuto. Perder el lake al reiniciar el runtime la noche antes de la entrega es un fallo evitable.

Particionado

Dos disposiciones para los mismos datos:

```
lake/curated/features/ticker=NVDA/date=2026-09-14/part-0000.parquet
lake/curated/features/date=2026-09-14/ticker=NVDA/part-0000.parquet
```

La primera favorece "dame toda la historia de NVDA" (partition pruning por ticker). La segunda favorece "dame todos los tickers de esta fecha", que es exactamente el patrón del ranking semanal cross-sectional. Con 10 tickers y particiones diarias, un año genera ~2.500 directorios con ficheros de pocos KB: es el **small-files problem**. Spark paga sobrecoste de planificación y apertura de ficheros mucho mayor que el tiempo de lectura. Mitigaciones: particionar por `year / month` en lugar de `date`, usar `coalesce / repartition` antes de escribir, o compactar con `OPTIMIZE` si usáis Delta. Medid y reportad: tiempo de lectura y número de ficheros antes y después.

6. Requisitos mínimos (para aprobar)

- Pipeline de ingesta funcional desde al menos dos fuentes distintas.
- Datos persistidos en Parquet o Delta con al menos un nivel de particionado justificado.
- Procesamiento en PySpark que produzca la tabla curated de features.
- Informe de calidad de datos con métricas reales, no un `print("ok")`.
- `baseline_score` calculado para los 10 tickers del universo, con pesos documentados.
- Top 3 por horizonte para `1W`, `1M`, `3M`, `6M` con probabilidad.
- Endpoint `GET /api/predictions` devolviendo JSON que pasa la validación de la plataforma, con `model_version: "v1"`.
- README con decisiones tomadas y por qué.

7. Opcional / bonus

- Delta Lake con time travel y demostración de reproceso de una partición.
- Ingesta incremental con control de watermark en lugar de full refresh.
- Ingesta de texto (noticias, transcripts) y un sentiment score sencillo.
- Datos macro y un ajuste del score por régimen de mercado.
- Tests automatizados de las transformaciones.
- Comparación empírica de dos esquemas de particionado con tiempos medidos.
- Contenedorización del pipeline.

8. Reto analítico: el Baseline Investment Score

El score es un **perceptrón simplificado**:

$$\text{score} = \sum w_i \cdot x_i + \text{bias}$$

Estructura de ejemplo:

$$\begin{aligned} \text{Investment Score} = & w_1 \cdot \text{valuation} + w_2 \cdot \text{growth} + w_3 \cdot \text{profitability} \\ & + w_4 \cdot \text{financial_health} + w_5 \cdot \text{momentum} \\ & + w_6 \cdot \text{earnings} + w_7 \cdot \text{competitive_position} \end{aligned}$$

Cadena conceptual: **input features** → **weights** → **bias** → **score** → **ranking** → **decision**. En RA1 los pesos los ponéis a mano. En RA2 los pesos los pone el dato. Ese es el puente exacto hacia el Machine Learning: la arquitectura no cambia, cambia quién decide w_i .

Normalización: por qué z-score y percentile rank no son intercambiables

Con 10 tickers, si uno cotiza a P/E 180 y el resto entre 15 y 35, el z-score comprime a los nueve normales en un rango estrecho y el score queda dominado por un solo outlier. El **percentile rank** es robusto a colas: el outlier ocupa el puesto 10 y no arrastra a los demás. El coste es que pierde magnitud (la diferencia entre P/E 15 y 16 pesa igual que entre 35 y 180). El dataset de referencia usa percentile rank en $[0,1]$ por este motivo. Decidid y justificad; una respuesta razonada vale más que copiar.

Recordad invertir el sentido donde toque: en valoración, un P/E bajo es bueno, así que el percentil debe calcularse de forma que el barato puntúe alto.

Baseline de referencia del dataset

Solo como ejemplo. En RA1 debéis **elegir y justificar vuestros propios pesos**:

$$\begin{aligned} \text{baseline_score} = & 0.20 \cdot \text{valuation} + 0.20 \cdot \text{growth} + 0.20 \cdot \text{quality} \\ & + 0.15 \cdot \text{financial_health} + 0.15 \cdot \text{momentum} + 0.10 \cdot \text{earnings} \end{aligned}$$

donde cada componente es un percentile rank cross-sectional en $[0,1]$.

De score a probabilidad

El contrato exige **probability**, y un score de ranking **no es una probabilidad**. Un **baseline_score** de 0.82 no significa 82% de probabilidad de hacer +10%.

Las tasas base reales medidas en el dataset de referencia son:

HORIZONTE	HIT RATE OBSERVADO
1W	≈ 9%
1M	≈ 35%
3M	≈ 43%

HORIZONTE	HIT RATE OBSERVADO
6M	$\approx 42\%$

Llegar a +10% en una semana es raro. **Quien declare 70% de probabilidad en un pick de 1W tiene garantizado un Brier score terrible.** Esta es una de las lecciones centrales del curso.

Enfoque concreto y suficiente: anclad la probabilidad en la tasa base del horizonte y dejad que el score module alrededor de ella con una transformación monótona acotada. Por ejemplo, con s = score normalizado en $[0,1]$ y $base_h$ la tasa base del horizonte:

```
# k controla cuánto os separáis de la tasa base; empezad conservadores (k = 0.5)
p = base_rate[h] * (1 + k * (s - 0.5) * 2)
p = min(max(p, 0.01), 0.95)
```

Con $base_rate["1W"] = 0.09$ y $k = 0.5$, el mejor pick de 1W sale en ~ 0.135 y el peor en ~ 0.045 . Rango estrecho, y correcto: es lo que dice el dato.

Advertencia explícita: **un score bien ordenado con probabilidades mal escaladas produce buen hit rate y Brier score pésimo.** El ranking y la calibración se evalúan por separado y podéis acertar en uno y fallar en el otro.

9. Cómo se os puntúa

La plataforma calcula por alumno: Resolved predictions, Hit Rate, Average/Median/Best/Worst Return, Average Alpha (contra un benchmark equiponderado), Average Probability, **Brier Score**, **Brier Skill Score**, **Calibration** y **Consistency**.

- **Brier score:** media de $(p - y)^2$, con $y=1$ si hit. Más bajo es mejor.
- **Brier Skill Score:** $1 - brier / brier_reference$, donde la referencia siempre es la tasa base de la cohorte para ese horizonte. Positivo = vuestras probabilidades aportan información. **Negativo = son activamente engañosas**, que es donde acaba el exceso de confianza.
- **Calibration:** se agrupan las predicciones en 10 bins de probabilidad y se compara la probabilidad declarada con la tasa de acierto observada.
- **Leaderboard score (0–100)** = 30% hit rate + 25% calibration + 25% relative return + 10% consistency + 10% sample reliability. Cada componente se mide **relativo a la cohorte**, no contra una constante absoluta.
- Se exigen **mínimo 12 predicciones resueltas** para aparecer en el ranking principal.

10. El dataset de referencia

En `data/mock/` del repositorio de la plataforma, exportado en CSV y JSON: `companies.csv`, `market_data.csv`, `fundamentals.csv`, `earnings.csv`, `competitors.csv`, `news_signals.csv`,

macro.csv , stock_features.csv , predictions.csv , prediction_results.csv , cycles.csv , students.csv .

Universo de 10 tickers: AAPL, MSFT, NVDA, META, GOOGL, AMZN, TSLA, JPM, V, NFLX. Es sintético pero internamente consistente: un mayor `revenue_growth` produce realmente un `growth_score` mayor. Existe para que diseñéis y probéis antes de que vuestro pipeline genere datos.

No enviéis predicciones basadas en este dataset sintético. Es solo para desarrollo.

11. Entregables

1. Repositorio con el código de ingesta y procesamiento.
2. Estructura del Data Lake documentada (árbol de directorios real, no dibujado a mano).
3. Informe de calidad de datos con métricas.
4. Notebook o script del cálculo del `baseline_score` .
5. Documento de decisiones: fuentes elegidas, campos, particionado, normalización, pesos, mapeo score→probabilidad. Con el "por qué" de cada una.
6. Endpoint `GET /api/predictions` desplegado y accesible, sirviendo `v1` .
7. Evidencia de al menos un payload validado correctamente por la plataforma.

12. Rúbrica de evaluación

CRITERIO	PESO	INSUFICIENTE	CORRECTO	EXCELENTE
Ingesta y fuentes	15%	Una sola fuente, no reproducible	Dos o más fuentes, script reproducible	Ingesta incremental, manejo de rate limits y reintentos, zona Raw íntegra
Data Lake y particionado	15%	Ficheros sueltos sin esquema	Parquet/Delta particionado y justificado	Particionado medido empíricamente, small-files gestionado
Procesamiento Spark	15%	Solo pandas	Transformaciones PySpark correctas	Uso consciente de Spark, esquemas explícitos, código modular y testeado
Calidad de datos	15%	Sin controles	Nulos, duplicados y rangos verificados	Además timezones, festivos, splits y dividendos tratados y documentados
Baseline Investment Score	20%	Pesos arbitrarios sin explicar	Features y pesos justificados, normalización coherente	Análisis de sensibilidad de pesos, tratamiento explícito de outliers

CRITERIO	PESO	INSUFICIENTE	CORRECTO	EXCELENTE
Score → probabilidad	10%	Se usa el score como probabilidad	Mapeo anclado en la tasa base del horizonte	Mapeo argumentado, probabilidades conscientes del 9% de 1W
Integración y documentación	10%	Payload rechazado o sin docs	Endpoint válido y README con decisiones	Documentación que permitiría a un tercero reproducir el pipeline

13. Errores comunes

- **Llamar a la API en cada ejecución** en lugar de cachear en el lake. Un día agotáis la cuota y no hay entrega.
- **Confundir score con probabilidad.** Provoca Brier Skill Score negativo.
- **Sobreconfianza en 1W.** La tasa base es 9%. Declarar 0.7 es un error medible.
- **Ignorar splits.** Un -90% falso en el retorno rompe todo lo que venga después.
- **generated_at sin offset.** Zod rechaza el payload completo y esa semana no puntuáis.
- **Rank duplicado dentro de un horizonte.** Mismo resultado: rechazo total.
- **Más de 12 predicciones.** Rechazo total.
- **Ticker en minúsculas.** Rechazo total.
- **Particionar por día con 10 tickers** sin compactar, y luego culpar a Spark de la lentitud.
- **Guardar solo los datos limpios** y descubrir un bug de parsing sin poder reprocesar.
- **Entrenar la intuición con el dataset sintético** y enviar esos picks como si fueran reales.

14. Formato de entrega

- Repositorio Git con historial de commits (no un único commit final).
- README en la raíz con: cómo ejecutar el pipeline, dependencias, fuentes usadas y decisiones de diseño.
- Dependencias declaradas de forma reproducible.
- El endpoint accesible públicamente por HTTPS, respondiendo en el ciclo de recogida.
- Documento de decisiones en Markdown dentro del repositorio.

15. Conexión con el Value Investing Challenge

La plataforma consulta vuestro endpoint una vez por ciclo, guarda la respuesta de forma inmutable con timestamp, y no la modifica jamás. Cuando el horizonte vence, calcula el retorno realizado y

resuelve la predicción. Por eso la regla de arriba no es retórica: **toda predicción queda registrada antes de conocer el resultado.**

Contrato del alumno

GET /api/predictions debe devolver:

```
{
  "student_id": "student-01",
  "model_version": "v1",
  "model_name": "Weighted Value Score",
  "generated_at": "2026-09-14T12:00:00Z",
  "predictions": [
    {
      "ticker": "META",
      "horizon": "1W",
      "rank": 1,
      "probability": 0.71,
      "expected_return": 0.14,
      "target_price": 712.40,
      "investment_thesis": "optional free text",
      "risks": "optional free text"
    }
  ]
}
```

Reglas de validación (Zod) aplicadas por la plataforma: `horizon` $\in \{1W, 1M, 3M, 6M\}$; `rank` $\in 1..3$ y único por horizonte; `probability` $\in [0,1]$; `expected_return` $\in [-1,10]$; `generated_at` debe ser un datetime ISO-8601 **con offset**; ticker en mayúsculas; sin tickers duplicados dentro del mismo horizonte; máximo 12 predicciones. **Un payload que falla la validación se rechaza por completo y esa semana el alumno no puntúa.** La guía completa está en `docs/STUDENT_INTEGRATION.md`.

En RA1 vuestro `model_version` es **v1**. Sus resultados quedan en el histórico y serán la referencia contra la que se medirá vuestro propio **v2** de RA2 y vuestro **v3** de RA3. El objetivo del curso no es ganar una semana: es poder responder con datos si el ML batió al scoring manual y si los agentes batieron al ML.

Every prediction must be recorded before we know the outcome. • PROJECT 1 — MARKET DATA LAKE